

Kino-PASTA – Kinodynamic Parallelized Accelerated Spaghetti Thread Agents

Alden Roswell, Owen Kranz, and Joseph Krauskopf

University of Colorado Boulder

alden.roswell@colorado.edu, owen.kranz@colorado.edu, joseph.krauskopf@colorado.edu

The authors grant permission for this report to be posted publicly.

GitHub: <https://tinyurl.com/38ub8uca>

Abstract—We introduce a novel approach to solving the kinodynamic motion planning problem via a decentralized multi-agent reinforcement learning algorithm called KinoPASTA. We introduce the motion planning problem and explain how planners have worked in the past. We then detail the inner workings of KinoPASTA and explain how it is different from typical planners. We then demonstrated the performance of KinoPASTA on typically benchmarks of environments with randomly generated scattered obstacles and environments with narrow passageways. KinoPASTA achieves a 97% success rate on narrow passage environments with mean time under 80ms on CPU. Rapidly exploring Random Trees (RRT) [1] is used as a benchmark and KinoPASTA outperforms on most marks.

I. INTRODUCTION

The field of motion planning solves a smorgasbord of interesting problems such as robotic manipulation and protein folding [2]. These problems all center around how to find a path to get a complex system from an initial state x_0 to a goal set X_{goal} without entering the set of obstacles X_{obs} which could be geometric and/or constraints on the system. For geometric systems, or systems that can be thoroughly controlled (e.g. holonomic systems like robotic arms), the motion planning community has created a variety of solvers that return trajectories in real-time [3]. Many of these planners utilize sampling-based motion planner which grow a tree from the initial configuration of the system until a node enters X_{goal} and the path back to the root node is returned as the solution. These planners have been successful and parallelization routines have greatly improved their performance.

Sampling based motion planner (SBMPs) are the life-blood of modern solutions. They work by building a tree where each node represents a configuration of the robot and each edge represents the path that the robot would take from one configuration to another. By starting with the starting configuration as the root node, sampling a random point in the space of all configurations ($\mathcal{C}_{free}$), extending the nearest node, and repeating, the tree is grown until a node is extended into the goal region.

SBMPs have strong algorithmic properties which we try to emulate via our KinoPASTA algorithm. The first is probabilistic completeness; all SBMPs, when continually run, will find the goal with a probability approaching 1, if a valid path exists. The next algorithmic property is that they are Voronoi biasing. This means that the growth of the tree is biased

towards regions which have not been explored thoroughly. What this means practically is that when a planner finds a new, unexplored area (such as when it just comes out on the other side of a narrow passage), the tree explodes from that new point.

SBMPs have deficiencies that we aim to address. First, is memory consumption, the tree growth in modern GPU based algorithms grows naïvely exponentially resulting in running out of memory quickly. While many approaches remedy this via sparse extension nodes [4], [5], SMDPs have a tendency to grow too dense of a tree when a sparse tree could do just fine. KinoPASTA strives to first sparsely covered the free area and then fill in from there. KinoPASTA also has linear growth of the ‘tree’ which will be discussed more later.

The motion planning problem has multiple different aspects which complicated the solution. These aspects present challenges that are still open research questions. We investigate a few of these in this paper. The first aspect of challenge is that of dimension. SBMPs tend to scale well with dimension, but advanced, GPU algorithms like KinoPAX [4] begin to break down when solving higher dimension problems. This limits their applicability to difficult problems such as humanoid robotic path planning and protein folding. Inherently, this is due to the fact that in order to reason over a tree to select nodes for parallel extension, one must discretize the configuration space, which grows exponentially with dimension (degrees of freedom of the system). KinoPASTA bridges this gap by maintaining massive parallelism without need for costly discretization. Another constraint is that the system must not violate its kinematic and dynamic constraints while going to the goal. This is accomplished by extending nodes of the tree via a propagation routine under chosen (often random) controls. This avoids the need to solve two-point boundary problem and guarantees that the resultant path is dynamically feasible. We ensure that KinoPASTA is capable of maintaining kinematic and dynamic constraint and it is therefore a ‘kinodynamic’ motion planner.

MOTION PLANNING PROBLEM

Let the **configuration space** be $\mathcal{C} \subseteq \mathbb{R}^n$, partitioned into:

- $\mathcal{C}_{free} = \mathcal{C} \setminus \mathcal{X}_{obs}$ — the free space
- $\mathcal{X}_{obs} \subset \mathcal{C}$ — the obstacle region
- $\mathcal{X}_{goal} \subset \mathcal{C}_{free}$ — the goal region

Algorithm 1: Generic Kinodynamic Tree-Based Planner

Input : $X, U, X_{\text{goal}}, X_{\text{obs}}, x_{\text{init}}, N$
Output: Valid trajectory $x_{1:T}$ if one is found

```

1  $G \leftarrow (\mathbb{V} \leftarrow \{x_{\text{init}}\}, \mathbb{E} \leftarrow \emptyset);$ 
2 for  $N$  iterations do
3    $x_{\text{rand}}, u_{\text{rand}} \leftarrow \text{Sample}();$ 
4    $n_{\text{select}} \leftarrow \text{Select}(x_{\text{rand}});$ 
5    $n_{\text{new}} \leftarrow \text{Extend}(n_{\text{select}}, u_{\text{rand}});$ 
6   if  $\text{ValidityCheck}(n_{\text{select}}, n_{\text{new}})$  then
7      $\mathbb{V} \leftarrow \mathbb{V} \cup \{n_{\text{new}}\};$ 
8      $\mathbb{E} \leftarrow \mathbb{E} \cup \{\text{edge}(n_{\text{select}}, n_{\text{new}})\};$ 
9 return  $G = (\mathbb{V}, \mathbb{E})$ 

```

Problem Definition

Given a start configuration $x_0 \in \mathcal{C}_{\text{free}}$, find a trajectory $x : [0, T] \rightarrow \mathcal{C}$ such that:

$$\begin{aligned}
 x(0) &= x_0 && \text{(initial condition)} \\
 x(T) &\in \mathcal{X}_{\text{goal}} && \text{(goal condition)} \\
 x(t) &\notin \mathcal{X}_{\text{obs}} \quad \forall t \in [0, T] && \text{(collision avoidance)}
 \end{aligned}$$

subject to the system dynamics:

$$\dot{x}(t) = f(x(t), u(t)), \quad \forall t \in [0, T]$$

where $u(t) \in \mathcal{U}$ is the control input and $\mathcal{U}$ is the admissible control set.

II. BACKGROUND

A. Sample Based Motion Planners

The traditional sampling based motion planners (SBMP) rely on stochastically sampling agent configurations, and growing a graph of valid configurations in the environment, then traversing that graph to find a valid motion path. The canonical example is the Rapidly Exploring Random Tree algorithm (RRT) [1], in which the root of a tree is placed at the starting configuration then random valid agent configurations are sampled and added to the tree, and upon the tree growing to the goal, the valid motion path can be extracted. RRT* [6] extends the original algorithm by presenting an asymptotically optimal algorithm by rewiring the original computed path. Both RRT and RRT* are probabilistically complete, meaning that given infinite time the algorithms are guaranteed to find a valid motion path if one exists.

B. GPU Planners

Recently, work has been done to improve the canonical SBMP algorithms by parallelizing their algorithms in an effort to allow for online kino-dynamic motion planning. By exploiting the parallelization potential of GPU's, massive computational efficiency can be implemented on the traditional SBMPs. Parallelized planners such as Kino-PAX [4] simultaneously expand tree nodes per each time step, massively improving the real time planning.

C. Multi-Agent Reinforcement Learning

Multi-Agent Reinforcement Learning (MARL) is a field of reinforcement learning that considers N agents acting in a shared environment, each working alone or together to maximize its cumulative reward [7]. In centralized training, decentralized execution (CTDE) [8], a central critic has access to the global state during training, but each agent simply has its own policy and local observations during execution. This architecture is useful for scenarios where agents don't share information and will act similar to a swarm.

D. Motion Planning as a POMDP

The motion planning problem can be phrased as a Partially Observable Markov Decision process (POMDP) [9] where the planner is trying to be optimized using a policy. Each agent has a belief over its environment via sensor measurements, and a learned policy maps those observations into actions. Prior work has been done to apply reinforcement learning into narrow-passage environments [10] and guiding SBMPs with learned heuristics [11]. These learning based approaches seem to be fragile to problem formulation. Additionally, none of these approaches attempt to design the learning architecture with parallelization and swarm behavior to solve the motion planning problem.

III. PROBLEM FORMULATION

We decided to formulate the motion planning problem as a POMDP where we defined an action and observation space to improve the planning efficiency of the agents. By condensing the large problem of motion planning into a more digestible format this allows the application of well developed reinforcement learning methodologies to create a motion planning policy. Below we define the POMDP:

State Space: The full environment state at time step t consists of: the set of obstacles, current poses of N agents, the backtrack buffer of N agents, and the position sum history of N agents.

$$\mathcal{S} = (\mathcal{X}_{\text{obs}}, \mathcal{X}_{\text{goal}}, \mathcal{T}_{\text{candidacy}}, (x_i, y_i, \theta_i), (x_i^{\text{prev}}, y_i^{\text{prev}}, \theta_i^{\text{prev}})), \quad i = 1, \dots, N$$

The obstacle layout is hidden from the policy, these obstacles are revealed via each agent observing ray casts.

Action Space:

Each agent has three possible actions:

- 1) **Continue (C)**
- 2) **Teleport (T)**
- 3) **Backtrack (B)**

$$A = \{C, T, B\} \tag{1}$$

and each always approximates its own **Teleport Candidacy (TC)**. The **continue** action is where the agent will randomly propagate control inputs to "continue" along a random walk path. The **teleport** action allows a given agent to set its state to that of any other agent, in a sense, teleporting to the current state of another agent. This agent is decided by randomly selecting from an elite subset

of the all of the agents based on their own estimated Teleport Candidacy values (with higher TC values meaning they are better targets to be teleported to). Finally, the **backtrack** action allows an agent to randomly return to a previous state along its path.

Observation Space: At each time step, each agent i receives a 7 dimensional body frame vector:

- Mean prior relative (dimension 2)
- Ray distances (dimension 3)
- Distance to goal (dimension 2)

$$o_i = [\delta_{\mu,i}^{\text{fwd}}, \delta_{\mu,i}^{\text{left}}, r_i^-, r_i^0, r_i^+, \delta_{g,i}^{\text{fwd}}, \delta_{g,i}^{\text{left}}] \in \mathbb{R}^7$$

The mean prior relative $(\delta_{\mu,i}^{\text{fwd}}, \delta_{\mu,i}^{\text{left}})$ is used to tell the agent how much progress it is making, i.e. if the agent is making forward progress, the mean prior relative will remain high, but if the agent is not making progress in the environment (backtracking a lot, stuck in a corner) the mean prior relative will decrease.

$$\delta_{\mu,i} = R(\theta_i)^\top \left(\frac{1}{t} \sum_{k=0}^t \begin{bmatrix} x_i^{(k)} \\ y_i^{(k)} \end{bmatrix} - \begin{bmatrix} x_i \\ y_i \end{bmatrix} \right)$$

Three ray casts (r_i^-, r_i^0, r_i^+) are captured for each agent i at each time step, these ray casts are 45 degrees separated, centered at the heading angle θ and return the distance to a wall or obstacle.

$$r_i^\phi = \frac{\min(\text{hit_distance}(\theta_i + \phi), r_{\max})}{r_{\max}}$$

The distance to goal metric $(\delta_{g,i}^{\text{fwd}}, \delta_{g,i}^{\text{left}})$ is computed by subtracting the agents current position from the goal's current position when rotated in agent i 's body frame.

$$\delta_{g,i} = R(\theta_i)^\top \left(\begin{bmatrix} x_g \\ y_g \end{bmatrix} - \begin{bmatrix} x_i \\ y_i \end{bmatrix} \right)$$

The combination of these three metrics allow each agent to understand the environment, and determine its progress to the goal.

Transition Function: The transition dynamics of agent i follow a Dubins' car model using Euler integration:

$$\theta_{mid} = \theta + 0.5\omega * dt \quad (2)$$

$$x_{k+1} = x_k + v \cos \theta_{mid} * dt \quad (3)$$

$$y_{k+1} = y_k + v \sin \theta_{mid} * dt \quad (4)$$

$$\theta_{k+1} = \theta_k + \omega_k * dt \quad (5)$$

Where $\omega_k \sim \mathcal{U}(\omega_{min}, \omega_{max})$ represents a randomized control input from the set with $\omega_{min} = -1.0$ and $\omega_{max} = 1.0$. v is fixed to 1.0 for simplicity.

Reward Function: The fitness of each episode is computed at the end of each rollout (a full simulation of the agents for T discrete timesteps) as the weighted sum of several reward terms:

$$R_{total} = R_{goal} + w_{goal} \cdot \frac{n_{reached}}{N} + w_{cov} \cdot C + w_{time} \cdot \frac{1}{n_{reached}} \sum_{i \in \mathcal{G}} \frac{1}{e_i^2}$$

where R_{goal} is defined with

$$R_{goal} = \begin{cases} 0 & \text{if } n_{reached} \geq 1 \\ -6 & \text{otherwise} \end{cases}$$

Where $n_{reached}$ is the number of agents that reached the goal region within the rollout time. $C \in [0, 1]$ is a coverage percentage and is the fraction of free space cells visited by any agent during the rollout. $\mathcal{G}$ is the set of agents that reached the goal and e_i is the number of forward steps agent i took to reach the the goal where backtrack steps are subtracted, which represents the path length to reach the goal (this gives some measure of smoother solutions being preferred). The weights $(w_{goal}, w_{cov}, w_{time})$ balance collective goal-reaching, environment exploration, and smoother paths to the goal.

The coverage term C is computed by placing a grid of cells with size Δc over the workspace, a cell is marked as visited whenever any agent's position falls within it. The reason to explore the environment is to allow for path diversification, thereby improving the chances of agents to find a path through the environment and not all simply follow a single route.

The weights used in training for $(w_{goal}, w_{cov}, w_{time})$ were 3, 6, 1 to highly weight the coverage of the algorithm since R_{goal} was tuned sufficiently so that algorithm prioritized finding at least one solution.

IV. METHODOLOGY

The structure of KinoPASTA is fundamentally different from most planners. Instead of growing a tree, we grow a bowl of pasta. We grow individual, strings of nodes called *spaghetti* from the root node that never branch and never communicate with each other. This structure presents a major benefits over a normal tree structure. Mainly, it makes the growth of the tree linear with iteration of the algorithm. This is a primary bottleneck of many of GPU planner such as Kino-PAX [4] which must dynamically change the number of nodes they extend to prevent exponential growth of the tree and exhaustion of memory resources. Before execution, one can compute the exact number of steps until a bowl of pasta runs out of memory. Another benefit is that if each *spaghetti* is truly independent of the others, then the algorithm can be lifted to a highly parallel architecture.

A major drawback of not having a tree based structure is that, in general, every node in the bowl of pasta may not be sampled infinitely often. There is no guarantee that the spaghetti will find unique paths and explore the entirety of the free space as time goes to infinity. This prevents the notion of probabilistic completeness, which is a key property of motion planning algorithms. Instead, we encourage probabilistic completeness properties via soft constraints. This encouragement is via rewarding coverage of the workspace highly. We argue that practically, no algorithm is able to run indefinitely and therefore, an algorithm that can find the goal quickly is often preferred and still has application.

Given the proposed structure, what algorithm could intelligently grow a bowl of pasta such that path(s) to the goal are found and the entirety of the set $\mathcal{C}_{free}$ is explored as the algorithm runs indefinitely? With the stated POMDP structure, we propose that each spaghetti thread in the bowl of pasta

should be controlled by an agent. This agent acts under the canonical reinforcement learning solution architecture: observe, act, step, learn, then repeat. We discuss the motivation of each step below:

Observe: The motion planning algorithm is inherently too complex to be fully observable. With high dimension configuration spaces (the search space), numerous obstacles that when mapped to configuration space are non-convex, highly unpredictable shapes, and huge tree structures, no algorithm has been able to ingest this information and solve motion planning outright. Instead, compromises are made. SBMPs typically utilize a simple nearest neighbor algorithm to 'observe' which nodes in the tree are promising. GPU SBMPs like KinoPAX utilize a costly configuration space discretizations to essentially 'observe' which nodes in the tree are in regions of poor exploration and hence, worth exploring more. Our observation structure as explained above gives each agent rough knowledge of its history, its surroundings, and the goal. At each iteration of the algorithm (where many iterations compose a full rollout), the observation vector for each agent is computed and fed to the policy.

Act: The act step is essentially a mapping from the observation to an action in our action step. In order to perform this mapping, we utilize a densely connected multilayer perceptron (MLP). This MLP takes in the full observation vector which flows through two hidden layers of 32 neurons each via *tanh* activation functions. The output layer has four neurons. The first three map to the action continue, backtrack, and teleport where the neuron with the largest value is chosen as the action. The fourth neuron is an approximation by the policy of how worth it is for other agents to teleport to that agent (Teleport Candidacy). When performing teleportation, the target agent is selected randomly among the top 10% of agents ranked by this Teleportation Candidacy's neuron's value. This policy is reused for all agents meaning this is a policy trained centrally but used in a de-central manner.

Step: This is performed by the environment which is assumed to be a generative function. The agents who chose continue are propagated forward under random control for a fixed timestep until they reach goal or hit an obstacle. The backtracking agents return to a previous position on their *spaghetti* up to ten spaces back. The teleporting agents are sent to one of the promising agent's current state. The reward is returned to the entirety of the agents only after the policy has done a full rollout in the environment (set to 250 timesteps though a wall-clock time budget could also be used).

Train: To train this central policy, we utilized a genetic based approach. Essentially, each generation, 20 random environments were generated. Some with random blocks spread apart (scattered) and some with narrow passageways. Example environments are shown in Figure 1. 100 random policies were then selected uniformly from all possible MLP parameters. Then, each policy was tested via a full rollout on the 20 random environments and rewarded accordingly after each rollout completed. In order to form the next generation of policies, a cross entropy method (CEM) was used. The top 20% of policies were selected based on their average reward received from the rollouts in all environments. A gaussian,

with dimensionality as large as the number of parameters in our policy MLP, was fit to these fittest policies. The next generation of policy is then sampled from this fit gaussian. This training process was repeated for 100 episodes and the best policy (based on the rollout in the random environments) was saved. A plot of the training curve is in 3.

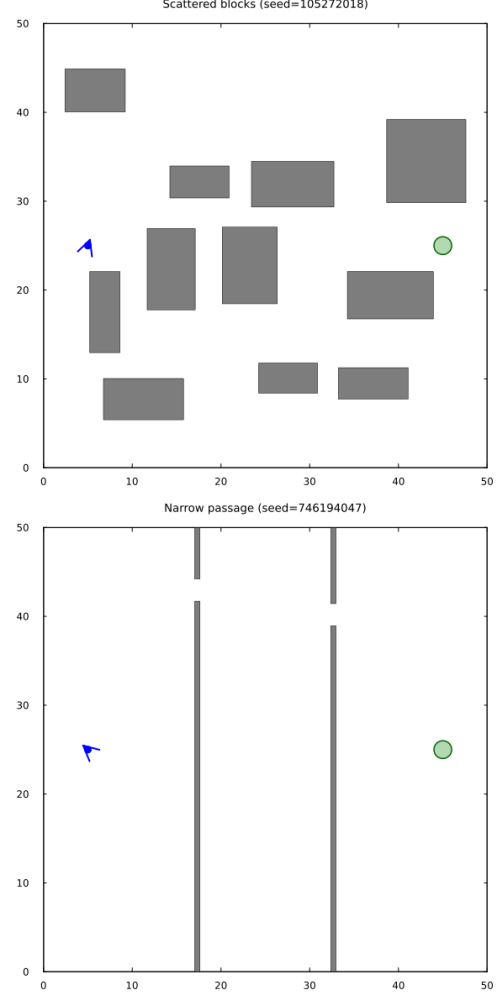


Fig. 1. Example training environments. One scattered the other with two narrow passage ways

V. RESULTS

We performed training of the policy over 100 generations with a population per generation of 64 agents. The training curve of this training is shown in Figure 3. Two example rollouts of the trained policy are then shown in Figure 2.

Additionally, the trained policy was evaluated on 100 environments of each class. The success rate (at least one solution found) and wall-clock average time to first solution (TTFS) are reported in Table I. RRT times are also reported. This benchmark was performed on a machine with an Intel i7-1355U processor with 16Gb of ram. A maximum planning time of 1 second was allowed.

CEM policy — random + narrow passage (77/1000 total reached)

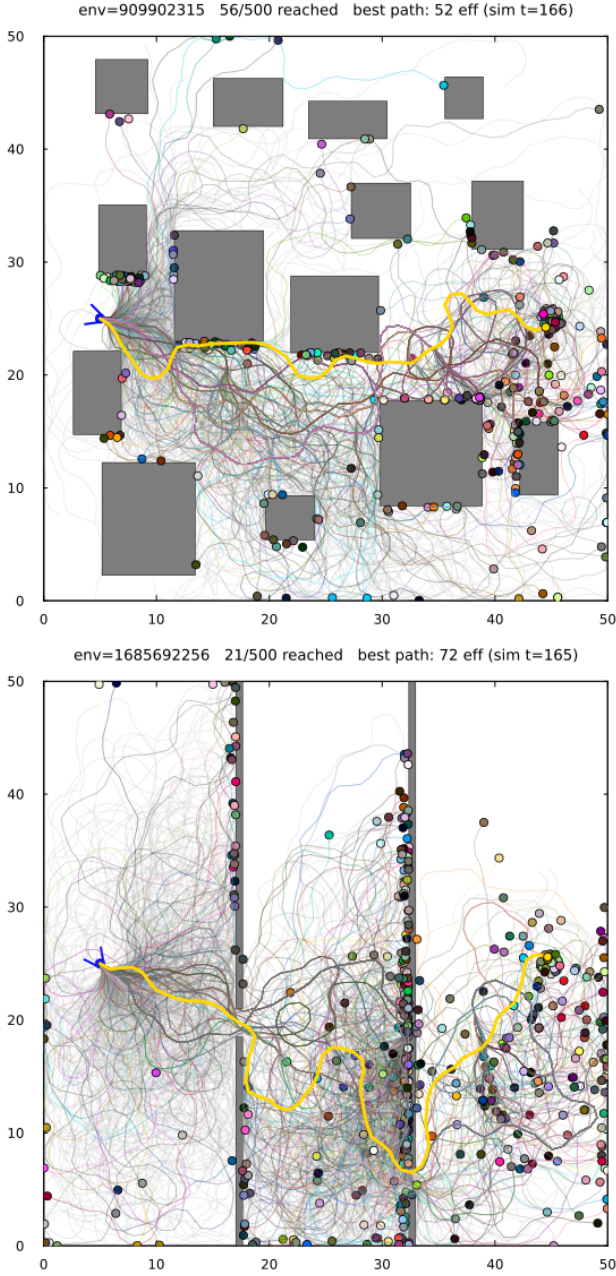


Fig. 2. Example rollouts with a trained policy on both an environment with scattered obstacles and an environment with two narrow passages

VI. DISCUSSION

KinoPASTA was able to achieve a near online performance metric, with average planning time of less than 30 milliseconds in scattered environments and just over 80 milliseconds for the narrow passage environments. This performance was achieved without any GPU implementation, meaning that additional performance improvements can be achieved. However, the problem would need to be expanded to a higher dimension and/or with more agents to warrant the development time of a GPU based implementation of KinoPASTA due to GPU

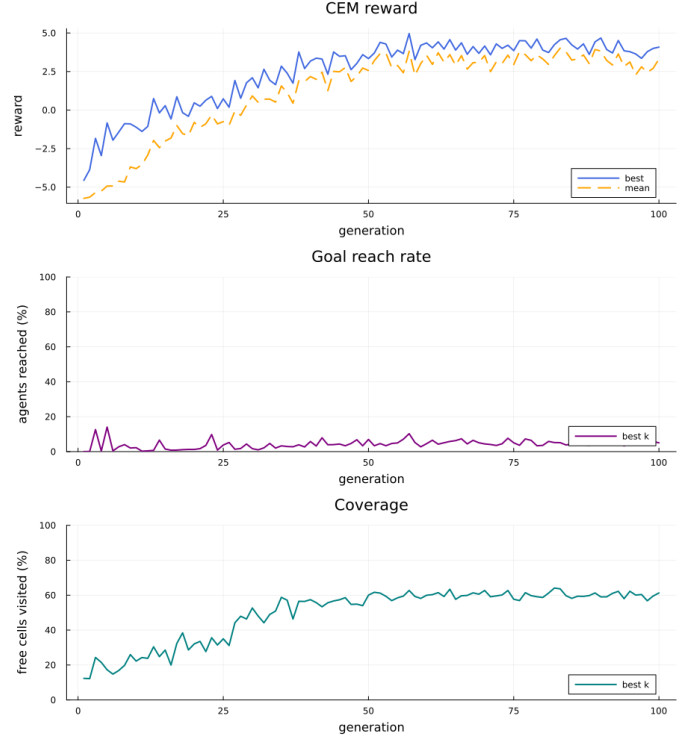


Fig. 3. Training data plotted as a function of generation of the CEM training. Both total reward and coverage continually increase. The goal reach rate stays low but stabilizes since this was weighted lower.

TABLE I
TRAINED CEM POLICY AND KINODYNAMIC RRT BASELINE EVALUATED OVER 100 RANDOMIZED ENVIRONMENTS OF EACH TYPE WITH A MAX PLANNING TIME OF 1 SECOND. SUCCESS MEASURED AS AT LEAST ONE OF $N = 500$ AGENTS REACHES THE GOAL (CEM) OR A TREE NODE ENTERS THE GOAL REGION (RRT). TTFS MEASURED AS THE WALL-CLOCK TIME FROM PLANNER START TO FIRST GOAL ENTRY, REPORTED AS MEAN $\pm$ STANDARD DEVIATION OVER SUCCESSFUL RUNS.

Method	Environment	Success rate	TTFS (s)
KinoPASTA	Scattered	100 / 100	0.0284 ± 0.0198
	Narrow	97 / 100	0.0808 ± 0.1568
Kinodynamic RRT	Scattered	100 / 100	0.0080 ± 0.0260
	Narrow	95 / 100	0.1113 ± 0.2032

launch overhead. The success rate was over 97% in both the narrow and scattered environments. This is quite significant as KinoPASTA is not probabilistically complete, yet is still able to achieve a very high success rate with a minimal implementation. With increased agents and planning time, it can be surmised that the success rate would approach 100%.

When we compare the performance of KinoPASTA to the canonical RRT algorithm. RRT outperforms KinoPASTA only in the time to first solution of the scattered environment. KinoPASTA was much faster in the narrow passage environments compared to RRT (30% speedup) and had less deviation in its time to first solution. It should obviously be stated that KinoPASTA is not generalizable, and a new environment shape that KinoPASTA had not trained on would likely result in RRT performing better. However, given the assumption that the environment distribution is known and could be adequately

trained on, we surmise that RRT would struggle more in more complicated environments, while KinoPASTA would be able to retain its performance due to its ability to learn the nuances of the environment.

There were multiple instances of emergent behavior we observed. First, we found that agents tended to backtrack more when near the goal. This behavior essentially created a controller than helped the agent get to the goal under its randomly sampled control action. If the random sampled control during the continue action did not improve the distance to the goal, then the agent tended to backtrack and try again under a new, random control.

Another emergent behavior was present on the narrow passage environment. We found that when an agent successfully passed through the gap, it would signal strongly to other agents that it was worthy as a teleportation target. Other agents would then quickly join in on the searching on the new side of the narrow passageway. As discussed earlier, this behavior is highly desirable for motion planning algorithms and is called Voronoi biasing.

VII. CONCLUSION

We have introduced KinoPASTA, a novel method of finding solutions to the motion planning problem. We have demonstrated a strong proof of concept that multi-agent policies can find solutions in random and narrow passage environments all while balancing the thorough exploration of the environment.

One natural extension of this work would be to parallelize the training routine and the rollout routine to run on the GPU. Both routines lend nicely to this. Each rollout of a prospective policy in a generation is inherently independent of the others and could easily be run on a GPU thread. The current training protocol implementation is already actually parallelized on CPU threads and the query of all the agents' policies for a timestep is performed by Flux.jl in a single batch. The execution of a policy with tens of thousands of agents after training would see the most benefit from a GPU parallelization schema. In this case, each agent would receive a thread on the GPU and could observe, act, and step inherently independent of the other agents. We expect that this would scale to tens of thousands of agents, drastically improving the quality of KinoPASTA solely by the volume of agents allowed to search in the environment.

Another extension of KinoPASTA would be on high dimensional systems. The observation space grows linearly with dimension and the action space remains fixed. Propagation may indeed become tricky in higher dimension due to the memory limits on a given GPU thread. This is in stark contrast to the KinoPAX algorithm which, due to its discretization protocol, does not scale well past approximately 12-16D systems [4]. Personally, we want to see this algorithm attempt the highest dimensional motion planning problems such as humanoid robotics and protein folding.

VIII. CONTRIBUTIONS

Work was split up equally. Brainstorming and problem formulation was performed by all and spearheaded by Owen.

AJ contributed heavily to the initial code development and tuning. Fine tuning training was performed by Owen and Joe. Main paper was written by everyone with Joe contributing to grammar and section drafting.

IX. ALGORITHM UNIQUENESS

All algorithms were implemented by us with help from AI coding agents thoroughly guided by us [12].

REFERENCES

- [1] S. M. LaValle, "Rapidly-exploring random trees: A new tool for path planning," *In*, vol. 129, 1998.
- [2] N. M. Amato, K. A. Dill, and G. Song, "Using motion planning to map protein folding landscapes and analyze folding kinetics of known native structures," in *Proceedings of the Sixth Annual International Conference on Computational Biology*, ser. RECOMB '02. New York, NY, USA: Association for Computing Machinery, 2002, p. 2–11. [Online]. Available: <https://doi.org/10.1145/565196.565198>
- [3] W. Thomason, Z. Kingston, and L. E. Kavraki, "Motions in microseconds via vectorized sampling-based planning," 2023. [Online]. Available: <https://arxiv.org/abs/2309.14545>
- [4] N. Perrault, Q. H. Ho, and M. Lahijanian, "Kino-pax: Highly parallel kinodynamic sampling-based planner," *IEEE Robotics and Automation Letters*, vol. 10, 2025.
- [5] Y. Li, Z. Littlefield, and K. E. Bekris, "Asymptotically optimal sampling-based kinodynamic planning," 2016. [Online]. Available: <https://arxiv.org/abs/1407.2896>
- [6] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," in *International Journal of Robotics Research*, vol. 30, 2011.
- [7] P. Hernandez-Leal, B. Kartal, and M. E. Taylor, "A survey and critique of multiagent deep reinforcement learning," *Autonomous Agents and Multi-Agent Systems*, vol. 33, 2019.
- [8] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Advances in Neural Information Processing Systems*, vol. 2017-December, 2017.
- [9] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, "Planning and acting in partially observable stochastic domains," *Artificial Intelligence*, vol. 101, 1998.
- [10] B. Chen, B. Dai, Q. Lin, G. Ye, H. Liu, and L. Song, "Learning to plan in high dimensions via neural exploration-exploitation trees," in *8th International Conference on Learning Representations, ICLR 2020*, 2020.
- [11] A. H. Qureshi, Y. Miao, A. Simeonov, and M. C. Yip, "Motion planning networks: Bridging the gap between learning-based and classical motion planners," *IEEE Transactions on Robotics*, vol. 37, 2021.
- [12] Anthropic, "Claude code," February 2026, cLI agent for terminal-based software engineering. [Online]. Available: <https://www.anthropic.com/claude-code>